



Veil

A Confidential MEV-Resistant CLOB DEX with Cross-Margined Lending on Zama FHEVM — The Complete Engineering Reference

PRODUCT SPEC · ARCHITECTURE · CODE WALKTHROUGH

Zama Developer Program Mainnet Season 3 · Builder Track · Composable Privacy Is the Key

sealed bid → encrypted clearing → per-user settle → vault collateral, never decrypted

Contents

PART I — THE THESIS

- 1 What Veil Is, and Why It Wins

PART II — THE ON-CHAIN SPINE

- 2 The Sealed-Bid Uniform-Price Batch Auction
- 3 ERC-7984 Escrow and Per-User Settlement
- 4 VeilLendingVault — Encrypted Health Factor
- 5 Vault ↔ CLOB Composition
- 6 VeilRegulatorRegistry, Deployment & Tests

PART III — THE OFF-CHAIN LAYER

- 7 The Off-Chain Clearing Solver
- 8 The Keeper Bot
- 9 Aggregate Public Decryption

PART IV — THE FRONTEND

- 10 The Design System
- 11 The Pages
- 12 The Components
- 13 The Wallet-Connected Trade Flow
- 14 UX Polish — Toasts, Onboarding, Mobile, Deploy

PART V — REFERENCE

- A The File Map
- B Deployed Addresses & Environment
- C The 3-Minute Demo Run-of-Show
- D Glossary

1

PART I — THE THESIS

What Veil Is, and Why It Wins

WHAT YOU WILL LEARN IN THIS CHAPTER

- The one-sentence thesis, and the MEV gap every other DEX leaves open
- The four composed primitives — CLOB, escrow, vault, audit — and how they chain
- The north-star loop: *seal* → *aggregate* → *clear* → *settle*, all encrypted end-to-end
- Why FHE is load-bearing, not decorative — what ZK and commit-reveal literally cannot do
- The current state: 5 contracts live and verified on Sepolia, hosted dashboard, multi-route end-to-end demo

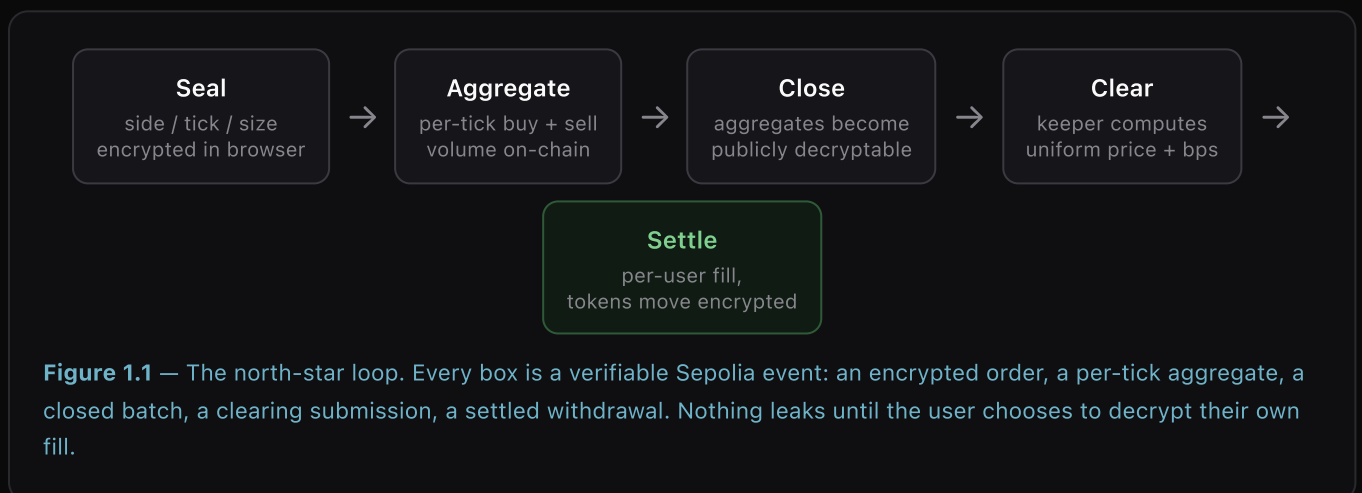
Every order book on Ethereum reveals state at match time. “Private” DEXes shipped to date use sequencer secrecy or commit–reveal latency — they just race to settle before the leak matters. Bots see your hand the moment it touches the chain, copy it, sandwich it, and front-run it. Lending protocols compound the problem: a public health factor is a public liquidation target. Privacy framed as a head start isn’t privacy — it’s a race the user always loses.

Veil closes that race. A sealed-bid uniform-price batch-auction CLOB where side, price tick, and size are encrypted end-to-end under FHE. Per-tick aggregate volumes accumulate as ciphertexts on-chain. After the batch closes the aggregates become publicly decryptable; any keeper computes the one clearing tick; per-user fills are computed under FHE and remain user-decryptable only. Stacked on top is a lending vault whose health factor is an `euint64` the contract acts on without ever decrypting, a regulator registry that lets users grant time-bounded delegated decryption for audit, and the composability moat: **vault collateral can be the source of a Veil sell escrow, and unfilled remainder returns to vault collateral on settle — with no decrypt in between.**

The four pieces

LAYER	WHAT IT IS	WHERE IT LIVES
CLOB	Sealed-bid uniform-price batch auction with per-tick FHE aggregates and pro-rata clearing	<code>VeilBatchAuctionV2.sol</code>
Escrow	ERC-7984 dual-token operator escrow on <code>placeOrder</code> , per-user <code>settle()</code> after clearing	<code>VeilBatchAuctionV2.sol</code> + <code>MockConfidentialToken.sol</code>
Lending	Encrypted collateral & debt, FHE health check, single-tx liquidation with no decrypt round-trip	<code>VeilLendingVault.sol</code>
Composition	Vault collateral as Veil sell escrow — encrypted state crossing contract boundaries	both contracts, via <code>escrowToVeil</code> / <code>placeOrderFromVault</code>
Audit	Time-bounded delegated-decrypt grants per account	<code>VeilRegulatorRegistry.sol</code>
Solver	Long-running Node loop closing batches, computing clearing, sweeping liquidations	Hardhat task <code>task:keeper:run</code>

The loop



The signature moment is the gap between “close” and “clear”: aggregates are publicly decryptable, the keeper computes the one clearing tick that maximises matched volume, and submits it. *Individual orders are never decrypted*. Fills are computed under FHE from the encrypted `(side, tick, size)` and the clearing parameters. A trader user-decrypts their own fill handle; no one else — not the keeper, not the contract owner — can read it.

WHY IT WINS THE RUBRIC

Season 3’s theme is “*Composable Privacy Is the Key*.” Veil maximises all four judging criteria at once: it uses ERC-7984 (the confidential token standard), the Zama React SDK (`useEncrypt`, `useUserDecrypt`, `usePublicDecrypt`), the FHEVM library in Solidity (`FHE.add`, `FHE.select`, `FHE.makePubliclyDecryptable`), and crucially — **encrypted state crosses contract boundaries**. A vault collateral `euint64` becomes a CLOB sell escrow without ever being decrypted. No ZK rollup and no commit-reveal scheme can do that.

Status

Live on **Ethereum Sepolia**: five Veil contracts deployed and source-verified (`VeilBatchAuctionV2`, `VeilLendingVault`, `VeilRegulatorRegistry`, `vWETH`, `vUSDC`). The dashboard is hosted at <https://veil-zama.vercel.app>, with five routes — landing, trade, vault, audit, and a no-tokens paper sandbox — all

wallet-connected and user-signed. Hardhat task pack drives the V2 lifecycle (faucet, close, clear, my-fill, settle) and a long-running `task:keeper:run` closes the loop unsupervised. The Foundry-style test suite covers V1 and V2 across 10 tests; vault tests are the remaining credibility gap, slated next.

The rest of this book walks the codebase from the chain up: the contracts that are the source of truth, the off-chain layer that drives them, and the frontend that turns four encrypted primitives into something a judge can watch happen.

2

PART II — THE ON-CHAIN SPINE

The Sealed-Bid Uniform-Price Batch Auction

WHAT YOU WILL LEARN IN THIS CHAPTER

- Why batch over continuous: the no-encrypted-divisor problem that kills price-time priority
- The discrete tick model and how a `uint8` tick index encrypts side + price intent
- How per-tick aggregates accumulate via `FHE.add` inside `placeOrder`
- The pro-rata bps trick: solving fairness at the marginal tick without dividing ciphertexts
- What `closeBatch` publishes, what `submitClearing` consumes

FHEVM blocks division by an encrypted divisor. That single fact rules out continuous matching for confidential limit orders: you cannot encrypt “the next available counterparty's price” and divide a fill against it. Batch auctions sidestep the problem by collapsing the matching question to a single computation per batch: *find the tick where supply meets demand*. Everything Veil does flows from this choice.

Four ticks, two orders

A batch in V2 has `NUM_TICKS = 4` discrete price levels: `tickPrice(t) = tickPrice0 + t · tickStep`. The current deployment uses `tickPrice0 = 3400` and `tickStep = 10`, so the four ticks are 3400, 3410, 3420, 3430 vUSDC per vWETH. An order is `(side, tickIdx, size)` where `side` is `ebool`, `tick` is `euint8`, `size` is `euint64` — all three encrypted at the browser, none readable on-chain.

```
// VeilBatchAuctionV2.sol — the order struct
struct Order {
    address trader;
    ebool isBuy; // true = buy, false = sell
    euint8 tickIdx; // 0..NUM_TICKS-1
    euint64 size; // demo: 0-decimal units to keep euint64 ceiling safe
    euint64 filledSize; // computed under FHE during submitClearing
    bool settled; // idempotency guard for settle()
    address marginVault; // 0 = wallet escrow; nonzero = vault composition (Ch. 5)
}
```

Per-tick aggregation under FHE

Every `placeOrder` loops over all `NUM_TICKS` per-tick aggregates and adds `size` to the matching one — or zero everywhere else. The trick is `FHE.eq` for the tick check, combined with `FHE.select` to gate the

addition:

```
function _aggregate(Batch storage b, ebool isBuy, uint8 tickIdx, uint64 size) private {
    for (uint8 t = 0; t < NUM_TICKS; t++) {
        ebool    match_    = FHE.eq(tickIdx, FHE.asEuint8(t));
        ebool    buyHere   = FHE.and(isBuy, match_);
        ebool    sellHere  = FHE.and(FHE.not(isBuy), match_);
        uint64    addBuy    = FHE.select(buyHere, size, FHE.asEuint64(0));
        uint64    addSell   = FHE.select(sellHere, size, FHE.asEuint64(0));
        b.buyVolume[t]     = FHE.add(b.buyVolume[t], addBuy);
        b.sellVolume[t]    = FHE.add(b.sellVolume[t], addSell);
        FHE.allowThis(b.buyVolume[t]);
        FHE.allowThis(b.sellVolume[t]);
    }
}
```

Cost: per order, `NUM_TICKS · (2 eq + 4 select + 2 add + 4 allow)`. With `NUM_TICKS = 4` and vialR-optimised Yul, the worst-case `placeOrder` fits inside the HCU budget with margin to spare even when escrow is layered on (Ch. 3) and the vault composition path is taken (Ch. 5). The frontend pins `gas: 18_000_000n` for direct placement and `20_000_000n` for the vault path.

Clearing — the pro-rata bps trick

After `closeBatch` publishes the aggregates (Ch. 9), a keeper computes the clearing tick off-chain and submits it back along with two bps fractions:

```
function submitClearing(
    uint256 batchId,
    uint8    clearingTick,
    uint16   marginalBuyBps,    // 0..10_000
    uint16   marginalSellBps   // 0..10_000
) external { ... }
```

For each order, `filledSize` is computed entirely in FHE from the encrypted side, tick, and size, and the *plaintext* clearing parameters. Orders strictly above the clearing tick on the buy side, or strictly below on the sell side, fill fully. Orders *at* the clearing tick fill pro-rata by the marginal bps:

```
// inside submitClearing, per order
ebool    tickEq    = FHE.eq(o.tickIdx, clearingEnc);
ebool    tickGt    = FHE.gt(o.tickIdx, clearingEnc);
ebool    tickLt    = FHE.lt(o.tickIdx, clearingEnc);
ebool    buyAbove  = FHE.and(o.isBuy, tickGt);
ebool    sellBelow = FHE.and(FHE.not(o.isBuy), tickLt);
euint64  fullFill  = FHE.select(FHE.or(buyAbove, sellBelow), o.size, zero);

ebool    buyAt     = FHE.and(o.isBuy, tickEq);
ebool    sellAt    = FHE.and(FHE.not(o.isBuy), tickEq);
euint64  buyMarg   = FHE.div(FHE.mul(o.size, uint64(marginalBuyBps)), uint64(BPS_DENOM));
euint64  sellMarg  = FHE.div(FHE.mul(o.size, uint64(marginalSellBps)), uint64(BPS_DENOM));
euint64  margFill  = FHE.select(buyAt, buyMarg, FHE.select(sellAt, sellMarg, zero));

o.filledSize = FHE.add(fullFill, margFill);
```

The bps divisor is plaintext (`BPS_DENOM = 10_000`), so `FHE.div(_, plain)` is legal. The bps fractions encode the matching ratio at the marginal tick — an honest keeper computes them so total filled buy equals total filled sell. A dishonest one would have to claim a clearing the publicly-decrypted aggregates don't support, which any second observer can refute.

ONE TRADE-OFF, NAMED HONESTLY

Continuous price-time priority is a casualty of this design — batch auctions intentionally flatten time priority within a window. Wilson (1979) and Budish, Cramton, Shim (2015) argue this is a feature, not a bug: continuous limit-order books are a race that latency arbitrageurs always win, while uniform-price batches reward genuine willingness to transact. Veil is the first deployment of that argument with end-to-end encrypted order intent. The cost is one cleared price per N blocks; the win is that no one — sequencer, MEV searcher, or block builder — sees the book.

ERC-7984 Escrow and Per-User Settlement

WHAT YOU WILL LEARN IN THIS CHAPTER

- The ERC-7984 operator model and why both `vWETH` and `vUSDC` are pulled on every order
- The `_pullEscrow` branching: `FHE.select` gates which side actually moves
- Why a buy escrow is sized at `maxTickPrice()` — the worst-case quote escrow
- Per-user `settle()` : filled side + unfilled refund + idempotency
- The `marginVault` per-order slot that threads the composition path through (Ch. 5)

Once an order is sealed and aggregated, the contract still has to hold the trader's collateral until clearing. Veil binds against the ERC-7984 confidential-token interface for that — the same slice OpenZeppelin's reference implementation exposes, the same slice the live Sepolia `cWETH/cUSDC` follow. The operator approval is one-time per token; after that the contract pulls escrow inside `placeOrder` with no further user signature.

The operator approval

ERC-7984 distinguishes *operators* from EOAs: a holder grants an operator the right to call `confidentialTransferFrom` on their behalf for a TTL. The dashboard's `ApproveRow` chip signs a `setOperator(VEIL_V2, now + 24h)` for both `vWETH` and `vUSDC`. After that, the user's wallet only ever signs `placeOrder` and `settle` — the token movements happen inside those transactions.

The dual-pull escrow

`placeOrder` doesn't know whether the order is a buy or a sell — `isBuy` is an `ebool`. So both tokens are pulled on every order, and `FHE.select` sets the irrelevant side to encrypted zero:

```

function _pullEscrow(ebool isBuy, uint64 size, address marginVault) private {
    uint64 zero          = FHE.asEuint64(0);
    uint64 baseEscrow    = FHE.select(isBuy, zero,                                size);
    uint64 quoteEscrow   = FHE.select(isBuy, FHE.mul(size, maxTickPrice()), zero);

    if (marginVault != address(0)) {                                           // composition path (Ch. 5)
        FHE.allowTransient(baseEscrow, marginVault);
        IVeilMarginVault(marginVault).escrowToVeil(msg.sender, baseEscrow);
    } else {                                                                    // direct wallet path
        FHE.allowTransient(baseEscrow, address(baseToken));
        baseToken.confidentialTransferFrom(msg.sender, address(this), baseEscrow);
    }
    FHE.allowTransient(quoteEscrow, address(quoteToken));
    quoteToken.confidentialTransferFrom(msg.sender, address(this), quoteEscrow);
}

```

A sell order encrypts (`baseEscrow = size`, `quoteEscrow = 0`) and the contract pulls `size` vWETH plus zero vUSDC. A buy encrypts the inverse — (`baseEscrow = 0`, `quoteEscrow = size · maxTickPrice()`). The buy quote escrow is sized at the **worst-case price**: it has to cover a fill at the highest tick (since the user's tick is encrypted and only the keeper knows the clearing). Any overpayment refunds on settle.

WAR STORY

Why `baseEscrow` isn't selected on the call itself

An earlier draft tried `FHE.select(isBuy, baseToken, vault)` at the call site to avoid the if-branch. That doesn't compile: contract types aren't ciphertexts and a plain `address` can't be selected by an `ebool` in EVM. The branch *has* to be plaintext — `marginVault` is a plaintext per-order field, set at order placement, read back at settle. The encrypted parts are the amounts.

Settle — one user, one transaction

After `submitClearing` writes the per-order `filledSize`, each trader can call `settle(batchId, orderId)` independently. The function is *permissionless* for the trader and gated by an idempotent flag:

```

function settle(uint256 batchId, uint256 orderId) external {
    Order storage o = _orders[batchId][orderId];
    if (msg.sender != o.trader)                revert NotOrderTrader();
    if (o.settled)                             revert AlreadySettled();
    Batch storage b = _batches[batchId];
    if (b.state != BatchState.Cleared)         revert BatchNotCleared();
    o.settled = true;

    uint64 clearingPrice = tickPrice(b.clearingTick);
    uint64 maxPrice      = maxTickPrice();
    uint64 filled        = o.filledSize;
    uint64 size          = o.size;

    // four FHE arithmetic branches, then select on the encrypted side bit
    uint64 sellQuotePayout = FHE.mul(filled, clearingPrice);
    uint64 sellBaseRefund  = FHE.sub(size, filled);
    uint64 buyBasePayout   = filled;
    uint64 buyQuoteRefund  = FHE.sub(FHE.mul(size, maxPrice), FHE.mul(filled,
clearingPrice));

    uint64 basePayout = FHE.select(o.isBuy, buyBasePayout, sellBaseRefund);
    uint64 quotePayout = FHE.select(o.isBuy, buyQuoteRefund, sellQuotePayout);

    if (o.marginVault != address(0)) { // composition path
        FHE.allowTransient(basePayout, address(baseToken));
        FHE.allowTransient(basePayout, o.marginVault);
        IVeilMarginVault(o.marginVault).creditFromVeil(msg.sender, basePayout);
    } else {
        FHE.allowTransient(basePayout, address(baseToken));
        baseToken.confidentialTransfer(msg.sender, basePayout);
    }
    FHE.allowTransient(quotePayout, address(quoteToken));
    quoteToken.confidentialTransfer(msg.sender, quotePayout);

    emit OrderSettled(batchId, orderId, msg.sender);
}

```

The math: a sell receives $\text{filled} \times \text{clearingPrice}$ in quote and gets $\text{size} - \text{filled}$ base refunded. A buy receives filled in base and gets the overpayment $\text{size} \times \text{maxPrice} - \text{filled} \times \text{clearingPrice}$ refunded in quote. `FHE.select` on the encrypted `isBuy` picks the right pair without revealing the side — an observer can't tell from the transaction whether the trader bought or sold.

4

PART II — THE ON-CHAIN SPINE

VeilLendingVault — Encrypted Health Factor

WHAT YOU WILL LEARN IN THIS CHAPTER

- Why a confidential lending vault needs a fundamentally different health model than Aave
- The `Position` struct and the lazy `_ensurePosition` pattern
- How `_maxBorrow` uses plaintext divisors to fit inside FHEVM's no-encrypted-divisor rule
- The clamp-to-zero pattern that makes over-borrows silently safe
- Single-tx liquidation via `FHE.gt + FHE.select` — no decrypt round-trip

Aave-style protocols compute a public health factor and let anyone liquidate when it drops below 1. Veil's vault holds collateral and debt as ciphertexts, so the health factor itself is encrypted. The contract still has to act on it — deposit, borrow, withdraw, liquidate — without ever decrypting it. The whole vault is an exercise in computing booleans under FHE and branching on them via `FHE.select`.

The position

```
struct Position {
    uint64 collateral; // vWETH posted
    uint64 debt;      // vUSDC borrowed
    bool exists;      // lazy init flag
}

mapping(address => Position) internal positions;
```

Parameters at deploy: `price = 3400` vUSDC per vWETH (plaintext, oracle-free for the demo), `ltvBps = 7500` (75%), `liquidationBonusBps = 500` (5%, unused in the simplified liquidation). The price is plaintext on purpose — an encrypted oracle adds two layers of complexity to the demo without changing the FHE story, and an Aave-style price feed would still be a plaintext read inside `_maxBorrow`.

The plaintext-divisor max-borrow

```
function _maxBorrow(uint64 collateral) internal returns (uint64) {
    uint64 collateralValue = FHE.mul(collateral, price); // uint64 *
    return FHE.div(FHE.mul(collateralValue, uint64(ltvBps)), uint64(BPS_DENOM));
}
```

Three operations, all FHE-legal: `mul(euint64, uint64)` is a scalar multiplication, `div(euint64, uint64)` is a scalar division with the plaintext on the right. The total HCU cost per call is roughly $365k + 365k + 715k \approx 1.4M$ — well inside budget for a transaction that also does an `FHE.le` check and an `FHE.select`.

Clamp-to-zero everywhere

Every position-changing function follows the same pattern: compute the request, check whether it would break an invariant under FHE, select between the request and encrypted zero. Over-borrow, over-withdraw, and ill-typed inputs all become silent no-ops — the user gets a successful transaction with no state change, instead of a revert that would leak which invariant tripped.

```
function borrow(externalEuint64 amountExt, bytes calldata proof) external {
    Position storage p = _ensurePosition(msg.sender);
    euint64 requested = FHE.fromExternal(amountExt, proof);

    euint64 maxBorrow          = _maxBorrow(p.collateral);
    euint64 newDebtRequested    = FHE.add(p.debt, requested);
    ebool    fits              = FHE.le(newDebtRequested, maxBorrow);

    euint64 actual = FHE.select(fits, requested, FHE.asEuint64(0)); // clamp
    p.debt          = FHE.add(p.debt, actual);

    FHE.allowTransient(actual, address(debtToken));
    debtToken.confidentialTransfer(msg.sender, actual);
    _grant(p.debt, msg.sender);
    emit Borrowed(msg.sender);
}
```

If the user borrows above their LTV, `fits` is `false`, `actual` is encrypted zero, the transfer moves zero vUSDC, and the debt position is unchanged. The transaction still succeeds — on-chain it's indistinguishable from a successful small borrow. The trader user-decrypts their debt handle to see what actually happened.

WAR STORY

The first deposit that reverted

An early version of `withdraw`, `repay`, and `liquidate` reverted with a `NoPosition` custom error when called before any deposit. That meant the very first user click on `/app/vault` — before they'd ever deposited — threw a generic MetaMask "transaction likely to fail" warning. The fix was one line per function: replace the revert with a call to `_ensurePosition(msg.sender)`, which lazily initialises an empty `Position` with encrypted zero collateral and zero debt. After that, `withdraw` on a fresh account just no-ops cleanly — consistent with the clamp-to-zero pattern everywhere else, no more surprise reverts.

Single-tx liquidation

The cleverest part is liquidation. A naive design would: (1) compute `unhealthy` as an encrypted boolean, (2) decrypt it via a relayer round-trip, (3) if true, seize collateral in a follow-up transaction. Three transactions, two block delays, and the `unhealthy` bit is publicly visible during the round-trip — defeating the point.

Veil collapses it to one transaction with no decrypt:

```

function liquidate(address borrower) external {
    Position storage p = _ensurePosition(borrower);

    euint64 maxBorrow = _maxBorrow(p.collateral);
    ebool    unhealthy = FHE.gt(p.debt, maxBorrow);

    euint64 seizedCollateral = FHE.select(unhealthy, p.collateral, FHE.asEuint64(0));
    euint64 clearedDebt      = FHE.select(unhealthy, p.debt,      FHE.asEuint64(0));

    p.collateral = FHE.sub(p.collateral, seizedCollateral);
    p.debt       = FHE.sub(p.debt,      clearedDebt);

    FHE.allowTransient(seizedCollateral, address(collateralToken));
    collateralToken.confidentialTransfer(msg.sender, seizedCollateral);
    _grant(p.collateral, borrower);
    _grant(p.debt,      borrower);
    emit Liquidated(borrower, msg.sender);
}

```

If `unhealthy` is true, the keeper sweeps the full collateral and the debt is wiped to zero in the same FHE statement. If it's false, both seize and clear are encrypted zero — the transaction succeeds and changes nothing. The keeper can call `liquidate` on every known borrower every block; only the unhealthy ones produce real state changes, and which-was-which is invisible to anyone who didn't do the FHE.

5

PART II — THE ON-CHAIN SPINE

Vault ↔ CLOB Composition

WHAT YOU WILL LEARN IN THIS CHAPTER

- Why this chapter is the Season 3 moat: encrypted state crossing contract boundaries
- The vault's ERC-7984-style operator interface and the `escrowToVeil` entry point
- How `placeOrderFromVault` branches V2's escrow source without changing the order semantics
- The settle return path: `creditFromVeil` credits unfilled remainder back to vault collateral
- Why both sides of the round-trip happen without a single decrypt

Up to here the four primitives compose only weakly — they share a token but not state. This chapter is where they chain: a user's encrypted vault collateral becomes a Veil sell escrow without ever being decrypted, and the unfilled remainder returns to vault collateral on settle — also without a decrypt. The encrypted state of one contract flows directly into the encrypted state of another. This is what Season 3's theme literally means.

The vault, now an ERC-7984-ish counterparty

The vault grows three new function families: operator semantics, a Veil-only pull, a Veil-only push.

```

// VeilLendingVault.sol - additions
mapping(address => mapping(address => uint48)) internal _operatorUntil;

function setOperator(address operator, uint48 until) external {
    _operatorUntil[msg.sender][operator] = until;
}

function isOperator(address h, address s) external view returns (bool) {
    return _operatorUntil[h][s] >= block.timestamp;
}

/// @notice Veil pulls a sell-side base escrow out of the user's collateral.
function escrowToVeil(address user, uint64 amount) external returns (uint64) {
    require(_operatorUntil[user][msg.sender] >= block.timestamp, "not operator");
    Position storage p = _ensurePosition(user);

    ebool    canCover    = FHE.le(amount, p.collateral);
    uint64    capped     = FHE.select(canCover, amount, FHE.asEuint64(0));

    uint64    newColl    = FHE.sub(p.collateral, capped);
    uint64    maxAfter   = _maxBorrow(newColl);
    ebool    safe        = FHE.le(p.debt, maxAfter);
    uint64    actual     = FHE.select(safe, capped, FHE.asEuint64(0));

    p.collateral = FHE.sub(p.collateral, actual);
    FHE.allowTransient(actual, address(collateralToken));
    collateralToken.confidentialTransfer(msg.sender, actual); // vWETH lands at Veil

    _grant(p.collateral, user);
    FHE.allowThis(actual); FHE.allow(actual, msg.sender);
    emit MarginEscrow(user, msg.sender);
    return actual;
}

/// @notice Veil credits an unfilled remainder back into the user's collateral.
function creditFromVeil(address user, uint64 amount) external {
    require(_operatorUntil[user][msg.sender] >= block.timestamp, "not operator");
    Position storage p = _ensurePosition(user);
    FHE.allowTransient(amount, address(collateralToken));
    collateralToken.confidentialTransferFrom(msg.sender, address(this), amount); // Veil
ships vWETH back
    p.collateral = FHE.add(p.collateral, amount);
    _grant(p.collateral, user);
    emit MarginCredited(user, msg.sender);
}

```

Note the two clamps in `escrowToVeil` : first against `p.collateral` (can't lend more than you have), then against post-pull health (can't lend so much that your existing debt is now uncollateralised). Both clamps are `FHE.le` checks that select between the request and encrypted zero — same pattern as the vault's direct withdraw.

placeOrderFromVault on V2

V2 grows one new entry point and one one-shot authorize:

```
function placeOrderFromVault(
    externalEbool sideExt, externalEuint8 tickExt, externalEuint64 sizeExt,
    bytes calldata proof, address marginVault
) external {
    require(marginVault != address(0), "vault=0");
    _placeOrder(sideExt, tickExt, sizeExt, proof, marginVault);
}

/// One-shot grant so the vault can pull vWETH back from Veil on settle.
function authorizeMarginVault(address marginVault) external {
    require(marginVault != address(0), "vault=0");
    baseToken.setOperator(marginVault, type(uint48).max);
}
```

Under the hood `placeOrder` and `placeOrderFromVault` share an internal `_placeOrder` that takes a `marginVault` parameter (0 for direct, a vault address for composition). The aggregation logic is identical — the only difference is which contract pulls the base escrow in `_pullEscrow` (Ch. 3) and which is credited on settle. The `marginVault` address is stored per-order so settle can route the return path correctly.

The round-trip, with no decrypt

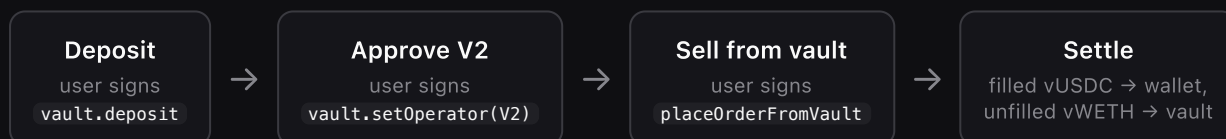


Figure 5.1 — The composition round-trip. The user's vWETH never leaves the FHE domain: it's pulled from encrypted vault collateral, escrowed in Veil's encrypted balance, partially filled (encrypted size), and the unfilled remainder is pushed straight back to encrypted vault collateral. No decrypt anywhere in the chain.

WHY NO OTHER PRIVACY PRIMITIVE CAN DO THIS

A ZK rollup with private orders has to decrypt at match time for the matching engine to function. Sequencer-secrecy chains decrypt the moment they sequence. Both leak. FHE's entire pitch is that the contract can compute on ciphertexts — so the matching engine, the operator check, the health gate, and the return credit are all just functions of encrypted handles, with the answers also encrypted. The composition isn't hard *because* FHE; it's the literal first thing FHE makes possible.

6

PART II — THE ON-CHAIN SPINE

VeilRegulatorRegistry, Deployment & Tests

WHAT YOU WILL LEARN IN THIS CHAPTER

- How time-bounded delegated decryption gives auditors access without putting it on-chain
- What `setRegulator` records and what `isAuditorOf` answers
- The Hardhat deploy pipeline: V2, vault (with post-deploy authorize), regulator, mock tokens
- Test coverage today: 10 V2 tests, the gap to close on the vault, and how composition is currently exercised

Veil's privacy story is complete only with a credible answer to the compliance question: *who gets to see this, and how is that controlled?* The regulator registry is the smallest possible answer — a directory of per-user time-bounded grants that the off-chain decryption flow consults. The contract is one struct, two writes, two reads, and 80 lines.

```

contract VeilRegulatorRegistry {
    struct Grant { address regulator; uint48 until; }
    mapping(address => Grant) internal _grants;

    event RegulatorSet(address indexed user, address indexed regulator, uint48 until);
    event RegulatorRevoked(address indexed user, address indexed regulator);

    function setRegulator(address regulator, uint48 until) external {
        require(regulator != address(0), "0");
        require(until > block.timestamp, "ttl");
        _grants[msg.sender] = Grant({regulator: regulator, until: until});
        emit RegulatorSet(msg.sender, regulator, until);
    }

    function revokeRegulator() external {
        Grant memory g = _grants[msg.sender];
        delete _grants[msg.sender];
        emit RegulatorRevoked(msg.sender, g.regulator);
    }

    function regulatorOf(address u) external view returns (address, uint48) {
        Grant memory g = _grants[u];
        if (g.until <= block.timestamp) return (address(0), 0);
        return (g.regulator, g.until);
    }

    function isAuditorOf(address u, address a) external view returns (bool) {
        Grant memory g = _grants[u];
        return g.regulator == a && g.until > block.timestamp;
    }
}

```

The registry is a directory; the actual decryption happens via Zama's delegated-decryption flow. The user signs an EIP-712 message granting the regulator's key the ability to decrypt their handles on the relayer, with an expiry. The registry mirrors that expiry on-chain so any contract or off-chain observer can ask "does X currently audit Y?" without ever seeing the decrypted data.

Deployment

Three Hardhat-deploy scripts: `deploy.ts` handles V1, `deploy-v2.ts` handles vWETH + vUSDC + V2, `deploy-vault.ts` handles vault + registry plus a post-deploy `authorizeMarginVault` call so the composition operator is live without manual intervention:

```
// deploy-vault.ts
const vault = await deploy("VeilLendingVault", { ... });
const reg    = await deploy("VeilRegulatorRegistry", { ... });

if (vault.newlyDeployed) {
    const veil = await hre.ethers.getContractAt("VeilBatchAuctionV2", v2Dep.address);
    const tx    = await veil.authorizeMarginVault(vault.address);
    await tx.wait();
}
```

Etherscan verification uses Sepolia's standard Etherscan API: `npx hardhat verify --network sepolia <addr>`. All five contracts source-verified after a clean deploy. The compiler is solc 0.8.27 with `vialR` enabled (FHE stack depth pushes the standard codegen past the limit on `submitClearing`).

Tests

The Foundry suite covers V1 and V2 in 10 tests: balanced clearing, buy-pro-rata, sell-pro-rata, the full view-and-revert path, and end-to-end `placeOrder` → `close` → `clear` → `settle` on the FHEVM mock. The vault tests are the credibility gap to close next — depositing, borrowing, `withdraw-blocks-under-debt`, the FHE.le clamp at the boundary, and the composition round-trip with `escrowToVeil` + `creditFromVeil`.

SANDBOX NOTE

The Sepolia keeper task and the front-end deployment have been exercised in concert — what they prove right now is contract-correctness across the happy path, not adversarial scenarios. The vulnerability sweep is its own line item: top of the list is `authorizeMarginVault`'s permissionless surface (anyone can grant any address operator status on V2's vWETH balance, which is sound under the operator-gated `escrowToVeil` but worth a second look), and the `_maxBorrow` intermediate value range under maximum-collateral inputs.

7

PART III — THE OFF-CHAIN LAYER

The Off-Chain Clearing Solver

WHAT YOU WILL LEARN IN THIS CHAPTER

- Why clearing is off-chain even though aggregates are public — the cost story
- The `computeClearing` algorithm: iterate ticks, find supply = demand, compute marginal bps
- Edge cases: no demand, no supply, perfect cross, fractional crosses
- How the keeper turns this into a single `submitClearing` call

The clearing computation is plaintext arithmetic over the publicly-decrypted aggregates. Putting it on-chain would be redundant — anyone can recompute it from the same public data — and expensive. The contract instead trusts the keeper to submit a clearing, and any honest second observer can refute a dishonest one by recomputing.

```
// tasks/Veil.ts - off-chain clearing
export function computeClearing(buyVol: bigint[], sellVol: bigint[]) {
  const NUM = buyVol.length;
  const BPS = 10_000n;
  let best = { tick: 0, buyBps: 10_000, sellBps: 10_000, matched: 0n };

  for (let c = 0; c < NUM; c++) {
    let demandAtOrAbove = 0n;
    for (let t = c; t < NUM; t++) demandAtOrAbove += buyVol[t];
    let supplyAtOrBelow = 0n;
    for (let t = 0; t <= c; t++) supplyAtOrBelow += sellVol[t];

    const matched = demandAtOrAbove < supplyAtOrBelow ? demandAtOrAbove : supplyAtOrBelow;
    if (matched > best.matched) {
      // pro-rata at the marginal tick
      const demandAbove = demandAtOrAbove - buyVol[c]; // strictly above
      const supplyBelow = supplyAtOrBelow - sellVol[c]; // strictly below
      const buyBpsNeed = matched - demandAbove > 0n
        ? Number((matched - demandAbove) * BPS / (buyVol[c] || 1n)) : 0;
      const sellBpsNeed = matched - supplyBelow > 0n
        ? Number((matched - supplyBelow) * BPS / (sellVol[c] || 1n)) : 0;
      best = { tick: c, buyBps: Math.min(10_000, buyBpsNeed),
        sellBps: Math.min(10_000, sellBpsNeed),
        matched };
    }
  }
  return best;
}
```

Iterate the candidate clearing tick from 0 to NUM_TICKS-1. At each candidate, total demand at or above (everything that would trade at this price or higher) and total supply at or below; matched volume is the min. The candidate that maximises matched volume wins. The marginal bps are the fraction of the candidate tick's buy or sell volume actually needed to balance the cross — computed as $(\text{matched} - \text{strictly-above-or-below}) \cdot 10_000 / \text{volume_at_tick}$, capped at 100%.

WAR STORY

An empty batch is a legal batch

Two early batches on Sepolia were closed empty — no one had placed orders. The first version of the keeper called `computeClearing` on all-zero aggregates and submitted `(tick=0, buyBps=10_000, sellBps=10_000, matched=0)`. That's harmless — the FHE settle math zeros everything out — but it cost a real transaction. The fixed keeper just skips submission when matched volume is zero. The contract still moves through `Closed` → `Cleared` only when there's something to clear; otherwise the batch stays `Closed` and the next open batch starts on schedule.



PART III — THE OFF-CHAIN LAYER

The Keeper Bot

WHAT YOU WILL LEARN IN THIS CHAPTER

- Why the keeper is a Hardhat task today, not a deployed Node service
- The two responsibilities: lifecycle (close + clear) and liquidation sweep
- The 12-second poll loop and the borrower-discovery via `PositionOpened` events
- How a single signer key drives both contracts atomically

`task:keeper:run` is a long-running Hardhat task that wakes every 12 seconds, ticks the V2 batch lifecycle, and optionally sweeps the vault for liquidations. It runs on the deployer key for now — one key, three responsibilities, no infrastructure. A deployed Render/Fly bot is the next iteration; the current setup ships and demos.

```
// tasks/Keeper.ts – the one-tick function
async function tickV2(hre: any, veil: any, signer: any) {
  const batchId = await veil.currentBatchId();
  const [, closeBlock, state] = await veil.getBatchState(batchId);
  const now = await hre.ethers.provider.getBlockNumber();

  if (Number(state) === 0 && now >= Number(closeBlock)) {
    const tx = await veil.connect(signer).closeBatch();
    await tx.wait();
  }

  const prev = batchId > 0n ? batchId - 1n : 0n;
  const [, , prevState] = await veil.getBatchState(prev);
  if (Number(prevState) !== 1) return; // already cleared, or no orders

  const NUM_TICKS = Number(await veil.NUM_TICKS());
  const buyVol: bigint[] = [];
  const sellVol: bigint[] = [];
  for (let t = 0; t < NUM_TICKS; t++) {
    const bh = await veil.getBuyVolume(prev, t);
    const sh = await veil.getSellVolume(prev, t);
    buyVol.push(bh === ZERO_HANDLE ? 0n : await fhevm.publicDecryptEuInt(FhevmType.euInt64, bh));
    sellVol.push(sh === ZERO_HANDLE ? 0n : await fhevm.publicDecryptEuInt(FhevmType.euInt64, sh));
  }
  const c = computeClearing(buyVol, sellVol);
  const tx = await veil.connect(signer).submitClearing(prev, c.tick, c.buyBps, c.sellBps);
  await tx.wait();
}
```

The same loop body runs as `task:keeper:tick` (single iteration, for cron-style runs) and `task:keeper:run` (forever). A `--liquidate true` flag adds a second pass that reads `PositionOpened` events from the vault to discover borrowers, then calls `liquidate` on each — the FHE.select clamp inside the vault means the call is a no-op for healthy positions, and only the genuinely unhealthy ones produce real seizures.

WHY ONE PROCESS, NOT A QUEUE

A production protocol with hundreds of borrowers would split liquidation across multiple keepers and add backoff. For the demo, one sequential process is sufficient and easier to reason about — Sepolia gas is free, the per-block discovery is cheap, and the worst case is a 12-second delay before a liquidation lands. The architecture admits more keepers trivially: the vault's `liquidate` is permissionless, so any keeper can sweep any subset of borrowers without coordination.



Aggregate Public Decryption

WHAT YOU WILL LEARN IN THIS CHAPTER

- The Zama public-decrypt flow: `FHE.makePubliclyDecryptable` → KMS → ciphertext
- How `closeBatch` marks the per-tick aggregates as publicly decryptable
- The frontend's `usePublicDecrypt` reading prior-batch aggregates into the UI
- Why the user-decrypt flow for individual fills is different — EIP-712, not public KMS

Aggregates are publishable; individual orders are not. Zama's protocol makes this distinction with two different permission models: `FHE.makePubliclyDecryptable(handle)` lets any client request the plaintext from the KMS, while `FHE.allow(handle, address)` grants an EIP-712-signed read to a specific address. Veil uses both: aggregates go public on close, fills stay private to the trader.

```
// VeilBatchAuctionV2.closeBatch
function closeBatch() external {
    Batch storage b = _batches[currentBatchId];
    if (b.state != BatchState.Open || block.number < b.closeBlock) revert BatchNotOpen();
    b.state = BatchState.Closed;
    for (uint8 t = 0; t < NUM_TICKS; t++) {
        FHE.makePubliclyDecryptable(b.buyVolume[t]);
        FHE.makePubliclyDecryptable(b.sellVolume[t]);
    }
    emit BatchClosed(currentBatchId);
    emit AggregatesPublished(currentBatchId);
    _openNewBatch();
}
```

The `AggregatesPublished` event is the keeper's signal that decryption can begin. Closing the batch also opens the next one in the same transaction — a clean rollover, no ceremony.

The frontend public-decrypt

The dashboard's `LastClearedPanel` component reads the prior batch's aggregate handles and pushes them through `usePublicDecrypt`, displaying the resulting plaintexts in a small per-tick table next to the live (still-encrypted) orderbook:

```
// components/veil/last-cleared.tsx
const { data: handles } = useReadContracts({ contracts: handleCalls });
const decrypt = usePublicDecrypt();

useEffect(() => {
  if (handleList.length === 0) return;
  decrypt.mutate(handleList, {
    onSuccess: (result) => {
      const next = Array.from({ length: NUM_TICKS }, () => ({ buy: 0, sell: 0 }));
      for (let t = 0; t < NUM_TICKS; t++) {
        const buyHandle = handles[t * 2]?.result as `0x${string}`;
        const sellHandle = handles[t * 2 + 1]?.result as `0x${string}`;
        next[t] = {
          buy: Number(result.clearValues[buyHandle] ?? 0n),
          sell: Number(result.clearValues[sellHandle] ?? 0n),
        };
      }
      setAggregates(next);
    },
  });
}, [handleList]);
```

Visually, this turns the orderbook from a perpetually-empty zero column into something that has a visible past: “here’s last batch’s depth at each tick, and where it cleared.” Without this, a first-time visitor sees zeros at every tick during the live batch (because the live aggregates are still encrypted) and assumes the app is broken.

User-decrypt for fills

Individual order fills go through a different path: when `submitClearing` writes `filledSize`, it follows it with `FHE.allowThis(o.filledSize)` and `FHE.allow(o.filledSize, o.trader)`. The trader can then call `useUserDecrypt` in the frontend, sign an EIP-712 message containing the handle and the contract address, and the relayer returns the plaintext. No one else — not the contract owner, not the keeper, not the KMS operators — can read it.

The Design System

WHAT YOU WILL LEARN IN THIS CHAPTER

- The Aurora theme — near-black, mono-font, one accent — and why it's the right register for confidential
- The `Cipher` and `Redacted` primitives that visually encode "encrypted"
- The panel system, the tabular-num discipline, and the encrypted-state aesthetic
- Why the app is webpack-served, not Turbopack — one stack's import quirk

A trading interface that doesn't feel like an instrument fails before it's read. The dashboard's look is deliberate: a near-black background with hairline borders and one accent green, mono fonts for every quantity, a perpetually-scrambling `Cipher` component that makes the "state is encrypted" promise visible. The aesthetic is sometimes called "Aurora" in the codebase because of the panel glow effect — a single radial gradient hinting at the encrypted depth underneath.

The palette

CSS custom properties in `globals.css`. Achromatic except for one accent and one warn:

TOKEN	ROLE
<code>--bg</code> / <code>--bg2</code> / <code>--bg3</code>	page / panel / inset background
<code>--text</code> / <code>--dim</code> / <code>--faint</code>	primary / secondary / tertiary text
<code>--line</code> / <code>--line2</code>	hairline borders / focus borders
<code>--accent</code> / <code>--glow</code>	the one accent, used for confirmations and emphasis
<code>--buy</code> / <code>--sell</code>	buy / sell sides — the only colours on the orderbook

Cipher and Redacted

Two primitives in `components/veil/primitives.tsx` carry the encrypted-state aesthetic. `Cipher` renders a perpetually-scrambling hex string that visually represents an encrypted value the user hasn't (yet) revealed. `Redacted` shows children when revealed, otherwise scrambles them with the same hex jitter. Both are SSR-safe via a stable placeholder and a `useEffect` that starts scrambling only on the client.

```
// primitives.tsx – the scrambling Cipher
export function Cipher({ len = 8, active = true }: { len?: number; active?: boolean }) {
  const placeholder = "0".repeat(len);
  const [txt, setTxt] = useState(placeholder);
  useEffect(() => {
    if (!active) { setTxt(placeholder); return; }
    setTxt(randStr(HEX, len));
    const id = setInterval(() => setTxt(randStr(HEX, len)), 70 + Math.random() * 60);
    return () => clearInterval(id);
  }, [active, len]);
  return <span className="veil-cipher" suppressHydrationWarning>{txt}</span>;
}
```

WAR STORY

Webpack mode, not Turbopack

@zama-fhe/sdk 's bundle uses the right-associative `**` operator that Turbopack's parser rejects. The fix isn't in the SDK — the operator is valid JS — it's in `package.json`: `"dev": "next dev --webpack"` and `"build": "next build"` (Next 16 only Turbopacks `dev` by default, so the build is already correct). Both scripts pin webpack and the SDK loads cleanly. Reverting to Turbopack saves nothing measurable in dev startup and breaks the FHE SDK — a forced choice that takes 30 seconds once you know and an hour if you don't.

Behind everything sits a single-line gradient glow on every panel (`veil-panel-glow`) and the same hairline border, the same 12px corner radius, the same subtle inset highlight on hover. The effect is instrument-panel without being claustrophobic. The orderbook's buy/sell columns are the only place the chromatic palette appears, which keeps red and green meaningful: every other state on screen is brightness.

The Pages

WHAT YOU WILL LEARN IN THIS CHAPTER

- Five routes, each doing one job in the demo story
- Why `/app` is the canonical trade view and `/app/paper` the no-tokens sandbox
- The vault and regulator routes as standalone product entry points
- The landing page's job: pitch first, link second

Five routes, each a distinct job: convince, trade, sandbox, lend, audit. The route collapse from V1/V2 to a single canonical `/app` happened mid-build — an earlier draft had `/app` as the paper sandbox and `/app/v2` as the real thing, which confused every first-time visitor including the project owner. The current layout is single-trade-route, with the paper sandbox accessible by direct URL for visitors who want to click around without setting operators.

ROUTE	PURPOSE
<code>/</code> landing	The thesis in one screen: a live encrypted batch panel, the four primitives, why FHE is required, CTAs to <code>/app</code> , <code>/app/vault</code> , and <code>/app/regulator</code> .
<code>/app</code> trade	The canonical CLOB. ApproveRow chips for vWETH and vUSDC, OperatorBar with reveal-balances, OrderBook live panel, OrderTicket with the "Use vault collateral" toggle, LastClearedPanel below the ticket, MyOrders list with per-row settle.
<code>/app/paper</code> sandbox	The no-tokens version of the CLOB. PlaceOrder only, no escrow, no settle — for first-time visitors who don't want to approve operators yet.
<code>/app/vault</code> lend	Confidential lending: operator approvals, PositionPanel with reveal, four ActionForm cards (deposit / withdraw / borrow / repay), inline glossary tooltips on LTV / utilization / health.
<code>/app/regulator</code> audit	Grant / revoke time-bounded delegated-decrypt access to a regulator address. Days-to-expiry input, current-grant display, single-tx writes.

The landing page leads with the encrypted-batch visual and the four-primitive layout — the rubric rewards a clear thesis up front, and the visual demonstrates the moat before the prose explains it. Every CTA points at `/app` first, with secondary chips to vault and audit. The paper sandbox is intentionally not advertised on the landing — it exists for testing, not for the first visitor.

The Components

WHAT YOU WILL LEARN IN THIS CHAPTER

- How the trade page composes ~10 panels into one coherent screen
- The OrderTicket and its VaultMarginToggle — the composition story in the UI
- The LastClearedPanel: the public-decrypt flow surfaced as a depth table
- The vault PositionPanel with inline-tooltip jargon
- The StartHere onboarding banner with localStorage-persisted progress

`TradeAppV2` in `components/veil/trade-app-v2.tsx` is the composition — almost 1000 lines, but every panel earns its line count. In order down the page: the StartHere onboarding banner (dismissable, four steps), the OperatorBar (two approve chips, two balance cells, a reveal button), the BatchPanel (orderbook + tick prices), and the right column with OrderTicket, LastClearedPanel, and MyOrders.

OrderTicket and the VaultMarginToggle

The OrderTicket is the central interaction: side toggle, tick selector, size input, submit. When the user picks “sell” the VaultMarginToggle appears underneath — a checkbox plus an inline paragraph explaining what “Use vault collateral” means, plus a contextual approve-vault button if the user hasn’t set V2 as their vault operator yet. When toggled on, the form’s submit routes to `placeOrderFromVault` instead of `placeOrder`.

```
// trade-app-v2.tsx (excerpted)
const txHash = composeSell
  ? await writeContractAsync({
      ...veilV2, functionName: "placeOrderFromVault",
      args: [side, tick, size, proof, VEIL_VAULT_ADDRESS as Address],
      gas: 20_000_000n,
    })
  : await writeContractAsync({
      ...veilV2, functionName: "placeOrder",
      args: [side, tick, size, proof],
      gas: 18_000_000n,
    });
```

LastClearedPanel

The component from Chapter 9 made visible. Six rows: tick, price, buy, sell, plus a totals row, plus a clearing-tick highlight. Public-decrypted via `usePublicDecrypt` on the prior batch's handles, polled every 10 seconds to catch the moment of close. The live (current) batch's columns stay zero in the OrderBook above — that's honest, because they're encrypted until close. The two panels together tell the truth: "here's last batch's depth, here's this batch's sealed activity."

PositionPanel + tooltips

The vault's headline component. Four cells: collateral (vWETH), debt (vUSDC), max-borrow, utilization — with inline tooltips on the labels that explain what utilization means and how the LTV ladder works. The reveal button drives `useUserDecrypt` on the two handles; until clicked, the cells show em-dashes. Below the cells: a Price / LTV / Health row, with a tone-aware Pill (healthy / at-risk) that switches automatically when utilization crosses 100%.

StartHere

Dismissable, localStorage-persisted, four steps that light up as the user completes them. The trade page's four steps: approve vWETH+vUSDC, reveal balances, place a sealed order, settle. The vault page's four: approve vWETH, approve vUSDC, deposit+borrow, reveal position. The progress is observed from existing app state (approval booleans, balance state, order count, settled count) so the banner is a pure projection — no extra event tracking, no parallel state.

The Wallet-Connected Trade Flow

WHAT YOU WILL LEARN IN THIS CHAPTER

- The wagmi v3 single-chain config and the local `WagmiSigner` shim
- The resilient `ConnectChip`: `connectAsync`, the 6-second watchdog, friendly errors under the button
- The `placeOrder` flow: encrypt → submit → receipt → parse event → update local order list
- The `settle` flow: user-decrypt the fill, submit settle, toast with Etherscan link and amount received

`lib/wagmi.ts` configures a single-chain wagmi v3 client: Sepolia, the injected connector (MetaMask / Rabby / Brave), and the public node RPC as transport. The frontend swaps `@zama-fhe/react-sdk/wagmi`'s `WagmiSigner` for a local `lib/zama-signer.ts` — the upstream package depends on a `watchConnection` export that wagmi v3 doesn't ship, and the local shim drops the `subscribe` step that needs it.

The resilient `ConnectChip`

The first version of the `Connect` button used wagmi's `useConnect().connect` with no error handling. The first time a user dismissed MetaMask's popup by closing the window (instead of clicking `Reject`), the mutation hung in `isPending` forever — the button stayed at "Connecting..." with no recovery. The rewrite uses `connectAsync` with `try/catch`, a 6-second watchdog that calls `reset()` if the mutation hangs, and surfaces the friendly error under the button so the user knows what to do:


```
// components/veil/connect-chip.tsx (core)
async function tryConnect() {
  setHint(null); setLocalBusy(true);
  try {
    await connectAsync({ connector: injected });
  } catch (err) {
    const friendly = formatError(err);
    console.error("connect rejected - raw:", err);
    console.error("connect rejected - friendly:", friendly);
    setHint(friendly);
    reset();
  } finally {
    setLocalBusy(false);
  }
}

useEffect(() => {
  if (!isPending) return;
  const id = setTimeout(() => {
    console.warn("Connect mutation stuck >6s - resetting wagmi state.");
    reset(); setLocalBusy(false);
  }, 6000);
  return () => clearTimeout(id);
}, [isPending, reset]);
```

The `formatError` helper digs through `shortMessage` → `message` → `details` → `cause` → `name` four levels deep, maps EIP-1193 error codes (4001 user rejected, -32002 request pending, 4901 wrong chain) to friendly sentences, and falls back to `JSON.stringify` with property lists so non-Error objects never print as `[object Object]`.

The placeOrder flow

From click to confirmed:

1. **Validate** — check approvals, check Sepolia network, switch if not.
2. **Encrypt** — `useEncrypt().mutateAsync({ values: [{value:isBuy, type:"ebool"}, {value:BigInt(tickIdx), type:"euint8"}, {value:BigInt(size), type:"euint64"}], contractAddress: VEIL_V2_ADDRESS, userAddress: address })`.
3. **Submit** — `writeContractAsync` with explicit `gas: 18_000_000n` so MetaMask doesn't pre-estimate and reject.
4. **Confirm** — `waitForTransactionReceipt`, parse the `OrderPlaced` event for the order index.
5. **Toast** — success toast with Etherscan link, prepend the order to the local `MyOrders` list with status "sealed".

The settle flow

An order row polls `getBatchState` every 4 seconds. When the batch transitions to `Cleared`, the row swaps its label to "Settle" and exposes a button. The button user-decrypts the fill (via `useUserDecrypt` on the `getOrderFill` handle), then calls `settle(batchId, orderIdx)` with `gas: 12_000_000n`. On receipt, a success toast reports the amount received with a clickable Etherscan link.

WAR STORY

The 15M-gas pin that wasn't enough

An early `placeOrder` pinned `gas: 15_000_000n`, which worked for the V1 paper contract. V2 added ERC-7984 escrow on both tokens plus the operator check plus the `FHE.allowTransient` pair, pushing the gas closer to 16M and tipping into "estimateGas" territory when the user's wallet was on the wrong chain. Bumping the pin to 18M for direct placement and 20M for composition fixed it everywhere — with margin to spare even if a future tick count or HCU price changes.

UX Polish — Toasts, Onboarding, Mobile, Deploy

WHAT YOU WILL LEARN IN THIS CHAPTER

- The toast system: tx-hash links to Sepolia Etherscan on every success
- `formatError` — raw viem chains in, friendly sentences out
- The composition explainer paragraph, the inline glossary tooltips, the settle success toast
- The mobile responsive sweep and what it touched
- The Vercel deploy: `veil-zama.vercel.app`, env vars per environment, alias to a stable subdomain

The first review of the live dashboard surfaced six gaps: no transaction-hash links in toasts, raw viem error chains shown to the user, no first-time onboarding, no explanation of the composition toggle, no success state on settle, no inline tooltips for jargon. All six got closed in a focused sweep; the result is a build a reviewer can run end-to-end without reading the README.

Toasts with Etherscan links

`components/veil/toast.tsx` exposes `useToast()` returning `{toast, flash}` and a `ToastView` at the bottom of every page. `flash` takes a `ToastInput` with `message`, `tone` (success / info / error), and optional `txHash` — if a hash is set, the toast renders an inline link to Sepolia Etherscan:

```
// components/veil/toast.tsx
{link && (
  <a href={link} target="_blank" rel="noreferrer noopener"
    className="font-[var(--font-mono)] text-[11px] text-[var(--accent)] hover:underline">
    {shortHash(toast.txHash)} ↑
  </a>
)}
```

formatError

`lib/format-error.ts` turns the gnarly viem nested-cause chain (*ContractFunctionExecutionError: The contract function "X" reverted with the following reason:...*) into one friendly sentence. It does three things in order: extract by EIP-1193 error code (4001 user rejected, -32002 already pending, 4901 wrong chain), match the raw message against a friendly-replacement table (insufficient funds, replacement underpriced, custom-error names), or fall back to the first revert-reason regex match. Non-Error objects are `JSON.stringify`d with property lists so they never print as `[object Object]`.

Inline composition explainer

The VaultMarginToggle now carries a paragraph under the checkbox: “Compose the lending vault into this trade. Your encrypted vWETH collateral is pulled directly from the vault and escrowed in Veil — your wallet vWETH balance is untouched. Unfilled size returns straight to vault collateral on settle. *This is the moat: encrypted state from one contract flowing into another, no decrypt in between.*” The italic is the ‘moat’ line because a reviewer might click without understanding why it’s special.

The mobile sweep

Six fixes touched ten components: vault PositionPanel `grid-cols-4` → `grid-cols-2 md:grid-cols-4`, three app headers got `flex-wrap` plus `min-h-16 py-2` with sub-md content hidden (the “Site” label, the ticker chip, the Sepolia badge), body padding tightened from `px-6` to `px-4 sm:px-6`, the BatchPanel footer stats got `flex-wrap` so the tick-prices string can drop a line, the Regulator current-grant grid stacks to `cols-1` below sm, and the landing Hero drops its `min-h-[72vh]` on phones so content stays above the fold.

Vercel deploy

The dashboard lives at **veil-zama.vercel.app**. Six `NEXT_PUBLIC_*` env vars synced to Production and Development (Preview required a Git- connected project, which the demo skipped). Aliased from the auto-generated `veil-ikjgqwhh9-oliens-projects.vercel.app` deploy URL via `vercel alias set <deploy> veil-zama.vercel.app`. Deployment Protection was disabled in the dashboard so the URL is genuinely public — otherwise reviewers hit a 401 authentication wall.

WHAT THIS CHANGES FOR THE SUBMISSION

Before the deploy: reviewers had to clone the repo, install dependencies, set five env vars, run two dev servers, and connect a wallet just to **see** the project. Five minutes minimum, fifteen if Node 25 throws hardhat warnings. After the deploy: open the URL, see the landing in two seconds, click Launch app and you’re wallet-prompting in five. The difference between “they read my README” and “they ran my product” is one Vercel URL.



PART V — REFERENCE

The File Map

A monorepo in three workspaces: the contracts (the truth), the keeper tasks (the off-chain driver), and the web app (the surface).

```

contracts/                                # Hardhat + @fhevm/solidity workspace
  contracts/
    VeilBatchAuction.sol                  # v1 paper CLOB (no escrow)
    VeilBatchAuctionV2.sol                # v2 CLOB with ERC-7984 escrow + settle + composition
    VeilLendingVault.sol                  # encrypted health factor, single-tx liquidation,
composition entry
    VeilRegulatorRegistry.sol             # time-bounded delegated-decrypt grants
    MockConfidentialToken.sol             # minimal ERC-7984 used for demo vWETH/vUSDC
    IConfidentialToken.sol                # the ERC-7984 slice Veil binds against
  deploy/
    deploy.ts / deploy-v2.ts / deploy-vault.ts
  tasks/
    Veil.ts                              # task:veil:status/close/clear/my-fill + computeClearing
    VeilV2.ts                             # task:veil-v2:status/faucet/close/clear/my-fill/settle
    Vault.ts                              # task:vault:address/seed/status/liquidate/my-position
    Keeper.ts                             # task:keeper:run/tick (closes + clears + sweeps
liquidations)
  test/
    VeilBatchAuction.ts                   # v1 tests
    VeilBatchAuctionV2.ts                 # v2 tests (10 passing on the FHEVM mock)
  hardhat.config.ts                       # Sepolia + local FHEVM mock + viaIR

web/                                       # Next.js 16 frontend (webpack mode)
  app/
    page.tsx                             # / landing
    app/page.tsx                         # /app real CLOB
    app/paper/page.tsx                   # /app/paper no-tokens sandbox
    app/vault/page.tsx                   # /app/vault confidential lending
    app/regulator/page.tsx               # /app/regulator audit grants
    providers.tsx                        # wagmi v3 + @zama-fhe/react-sdk providers
    globals.css                          # Aurora theme tokens + keyframes
  components/veil/
    trade-app-v2.tsx                     # /app composition (~1000 lines)
    vault-app.tsx / regulator-app.tsx
    orderbook.tsx                        # OrderBook + useBatchLifecycle
    last-cleared.tsx                     # LastClearedPanel (usePublicDecrypt)
    primitives.tsx                       # Cipher, Redacted, Pill, Btn, Wordmark, EthereumMark, Icon
    nav.tsx / connect-chip.tsx           # shared route nav + resilient wallet connect
    start-here.tsx / tip.tsx / glossary.ts
    toast.tsx                            # ToastView + useToast with tx-hash links
    sections.tsx                         # landing sections
  lib/
    abi-v2.ts / abi-vault.ts             # hand-curated ABIs
    use-v2-lifecycle.ts / use-veil-lifecycle.ts
    config.ts                            # env-driven contract addresses
    etherscan.ts / format-error.ts       # tx-hash links + error friendly-text mapping
    wagmi.ts / zama-signer.ts            # single-chain wagmi v3 + WagmiSigner shim

tokenops/                                # Mist – confidential disperse (TokenOps Special Bounty,
parallel)

```




PART V — REFERENCE

Deployed Addresses & Environment

Live on Ethereum Sepolia (chainId 11155111). Deployer:

`0x4599C71A28FA7D3C66c432bb95D2D9C3A671D16C`. All five contracts source-verified on Sepolia Etherscan. Hosted dashboard at <https://veil-zama.vercel.app>.

CONTRACT	ADDRESS
VeilBatchAuction (v1 paper)	<code>0xde5aC3708831BDd2DfDbF00614A2717f76eacb7e</code>
VeilBatchAuctionV2 (composition-aware)	<code>0x6F4eB2E35E91a1efC54c1927b1bdCcc667bC140F</code>
VeilLendingVault	<code>0x4A807C8bBB027e819d8Ab17bD22dF5b3C5876d2A</code>
VeilRegulatorRegistry	<code>0xD9EEF8C06cB520e3A423FC1043fBbf17D87d4A6</code>
vWETH (MockConfidentialToken)	<code>0xD7b35a8d7692B469d5C72D163e405C8c06177eCF</code>
vUSDC (MockConfidentialToken)	<code>0x5cb2dEe62375f3F9315BFE79B1551A5001593Cb8</code>

The vault state

PARAMETER	VALUE
<code>price</code>	3,400 vUSDC per vWETH (plaintext, oracle-free for demo)
<code>ltvBps</code>	7,500 (75% loan-to-value)
<code>liquidationBonusBps</code>	500 (5%, unused in simplified liquidation)
Seeded liquidity	10,000,000 vUSDC available to borrowers

Frontend environment

- `NEXT_PUBLIC_VEIL_ADDRESS` — V1 paper contract (for `/app/paper`)
- `NEXT_PUBLIC_VEIL_V2_ADDRESS` — V2 (canonical CLOB)
- `NEXT_PUBLIC_VEIL_BASE_ADDRESS` — vWETH
- `NEXT_PUBLIC_VEIL_QUOTE_ADDRESS` — vUSDC
- `NEXT_PUBLIC_VEIL_VAULT_ADDRESS` — lending vault
- `NEXT_PUBLIC_VEIL_REGULATOR_ADDRESS` — regulator registry
- `NEXT_PUBLIC_SEPOLIA_RPC` / `NEXT_PUBLIC_ZAMA_RELAYER_URL` — optional overrides; default to `SepoliaConfig` from the Zama SDK

Private signer keys live only in `~/config/hardhat-nodejs/vars.json`; the repo's `.env.local` contains nothing sensitive. Run the stack with `cd web && npm run dev` for local development (port 3000), or visit the hosted URL.

Contracts: `cd contracts && npx hardhat compile` then the keeper: `npx hardhat --network sepolia task:keeper:run --liquidate true`.



PART V — REFERENCE

The 3-Minute Demo Run-of-Show

The whole story in three minutes. The climax is beats 6–7: the composition round-trip where vault collateral becomes Veil escrow without a decrypt.

#	ON SCREEN	BEAT
0	Landing	"A confidential CLOB. Orders are encrypted in your browser, aggregated under FHE, cleared at one uniform price — with no one seeing the book."
1	/app, ApproveRow	One-time operator approval on vWETH and vUSDC — the ERC-7984 setup.
2	OperatorBar, reveal	Click Reveal balances. The user-decrypt flow pulls plaintext from the KMS — only you can see this.
3	OrderTicket, sell	Place a sell at tick 2. Side, tick, size all encrypted in the browser — the LastClearedPanel below shows what last batch looked like.
4	Keeper tab	The keeper closes the batch, public-decrypts the aggregates, computes clearing, submits. The orderbook's prior-batch column updates.
5	MyOrders, Settle	User-decrypt the fill; settle. Etherscan link in the toast shows the encrypted transfer.
6	/app/vault, deposit + borrow	Switch tabs, deposit 10 vWETH, borrow 5,000 vUSDC. Reveal position — collateral 10, debt 5,000, utilization 19.6%, health OK.
7	/app, "Use vault collateral"	Toggle on. Place a sell from vault collateral. The vault's encrypted vWETH becomes Veil's encrypted vWETH without a decrypt. Settle returns unfilled to vault collateral the same way.
8	/app/regulator	Grant a 30-day audit access to a regulator address. The compliance hatch, time-bounded and revocable.
9	Closing	"CLOB, escrow, lending, audit — four primitives, one composable encrypted state, every step a verifiable Sepolia event."

Reviewers can take the wheel after the demo: `/app` trades, `/app/vault` lends, `/app/regulator` grants audit. Every flow is user-signed end-to-end — no server keys, no hosted custody.



Glossary

TERM	DEFINITION
FHEVM	Zama’s Fully Homomorphic Encryption Virtual Machine: an EVM extension that lets Solidity compute on encrypted handles (<code>euint8</code> , <code>euint64</code> , <code>ebool</code>) without decryption.
HCU	Homomorphic Compute Units — FHEVM’s metering unit, separate from gas. Each FHE op (add, mul, select) has an HCU cost; the per-tx budget is 20M HCU with depth $\leq 5M$.
ERC-7984	The Confidential Fungible Token standard: <code>setOperator</code> , <code>confidentialTransfer</code> , <code>confidentialTransferFrom</code> , <code>confidentialBalanceOf</code> . Veil’s demo tokens implement it minimally.
Sealed-bid batch	A market microstructure that collects orders during a window with no mid-window matching, then clears them all at a single uniform price. Wilson 1979.
Uniform-price clearing	The one tick where supply meets demand — every filled order at that batch trades at the same price, regardless of where the trader bid.
Marginal bps	The fraction (0–10,000) of orders <i>at</i> the clearing tick that fill, encoded in plaintext to sidestep FHEVM’s no-encrypted-divisor rule.
Public decryption	The KMS-mediated flow that turns an <code>FHE.makePubliclyDecryptable</code> handle into plaintext anyone can fetch. Veil uses it on per-tick aggregates after close.
User decryption	The EIP-712-gated flow that lets a specific address read a specific handle. Veil uses it on per-order fills.
Operator (ERC-7984)	An address authorized to call <code>confidentialTransferFrom</code> on a holder’s behalf for a TTL.
Composition (Veil-specific)	The pattern where one contract’s encrypted state (vault collateral) becomes another contract’s encrypted state (Veil order escrow) without ever being decrypted.

seal → aggregate → clear → settle — every step a verifiable Sepolia event, every quantity an encrypted handle. That is Veil.